

The Pattern-Mapping Test as Standalone Operational Procedure: Verifying Hybrid Systems Composition by Mapping Each Adjacent Component to Exactly One Legitimate Pattern in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

— — —

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the *pattern-mapping test* — the operational procedure by which a deployment verifies that every adjacent AI component composed with a CKS substrate occupies exactly one of the three legitimate composition patterns the source paper supports — as a standalone procedure specification.

Abstract

The CKS pattern's hybrid systems composition commitment names three legitimate positions an adjacent AI component (RAG index, fine-tuned LLM, vector database, external knowledge store) may occupy relative to a CKS substrate: as input to a cell (Pattern A), as a derived view of substrate content (Pattern B), or as a separate concern (Pattern C). This note formalizes the deployment-level procedure that verifies the commitment by mapping each adjacent component to exactly one pattern using each pattern's sharpening properties as criteria. The test passes when every component maps unambiguously and exhaustively with the mapping documented as a substrate-resident rule authored by humans; it fails on pattern-confusion, multi-pattern mapping, unmappable partners, "fourth pattern" claims, and vendor- or framework-determined mappings. The note states the procedure, the violations it detects, the lifecycle moments at which it runs, and the limits of what it verifies.

1. Why the pattern-mapping test needs to be formalized as standalone

A separate note in this series formalizes the CKS pattern's commitment that hybrid deployments — CKS substrates composed with adjacent AI components such as RAG indexes, fine-tuned LLMs, vector databases, and external knowledge stores — must position each adjacent component in exactly one of three legitimate patterns: as input to a cell (Pattern A), as a derived view of substrate content (Pattern B), or as a separate concern (Pattern C). Companion notes formalize each pattern's sharpening properties and the three-patterns coherence claim — that the patterns are exhaustive, mutually distinguishable, and anti-pattern-excluding at the deployment level.

Joint formalization establishes the architecture; it does not produce a deployment-level procedure for verifying that the architecture holds in any particular deployment. The architectural commitment is per-component: each adjacent component must occupy exactly one pattern. Verifying it is therefore per-component as well — each adjacent component examined, the pattern it occupies identified, and the identification documented. This procedure has its own operational content, distinct from the pattern definitions, and any team running a CKS deployment must run it when adjacent components are added, removed, reconfigured, or migrated.

This note formalizes that procedure as the *pattern-mapping test*. The procedure is deliberately narrow: it verifies pattern mapping coherence and nothing else. Other tests in this series verify the five Requirements any composition partner must satisfy, the foundational architectural commitments at each partner, and reproducibility of the deployment's coordination state; the pattern-mapping test does not duplicate any of them. Naming this narrow scope precisely is what allows the test to be runnable, repeatable, and integrable with the rest of the verification workflow.

2. The architectural commitment under test

The commitment under test decomposes as follows.

The integrating frame. A hybrid deployment is CKS-coherent if and only if every adjacent AI component sits in one of three named patterns. Components occupying no named position have acquired an unnamed architectural role, and the commitments degrade where the role is unnamed.

Pattern A — adjacent component as input to a CKS cell. A cell consults the component during execution, reading from the substrate as source of truth and writing outputs back to the substrate under its orchestration rule. The component is part of the cell's reasoning environment; the substrate remains authoritative.

Pattern B — adjacent component as derived view of substrate content. Substrate content is indexed, embedded, or summarized into the component to support specific queries. The component is read as a derived view: not authoritative, regeneratable from substrate state at any time.

Pattern C — adjacent component as separate concern. The component handles a distinct concern that does not affect coordination state. The two systems coexist without architectural coupling.

The test verifies the integrating frame by mapping each adjacent component to exactly one of A, B, or C using each pattern's sharpening properties as criteria. A component satisfying multiple, none, or ambiguous patterns indicates a violation the test names — typically one of three named failure modes (substrate substitute, ungoverned writer, hidden bidirectional coupling) treated in §5.

3. Test procedure

The procedure has four operational steps.

Step 1 — Identify all adjacent AI components. Enumerate every AI component in the deployment that is not a CKS cell or substrate element: every retrieval index, every fine-tuned model, every vector database, every external knowledge store, every adjacent embedding or retrieval service. The enumeration must be exhaustive. Missing a component falsifies the test — the unnamed component is itself a canonical instance the test exists to detect.

Step 2 — Attempt to map each component to exactly one pattern using each pattern's sharpening properties.

Pattern A check. Does the component sit in a unidirectional cell-to-adjacent consultation flow under an orchestration rule — a cell reads the substrate as source of truth, queries the component for additional context, and writes outputs back to the substrate under the rule that authorized the write, with the component not writing substrate state directly? If yes, the component maps to Pattern A.

Pattern B check. Does the component sit in a unidirectional substrate-to-component derivation — its content is generated from substrate content through indexing, embedding, or summarization; it is not authoritative; conflicts with the substrate resolve in the substrate's favor; it is regeneratable from substrate state at any time? If yes, the component maps to Pattern B.

Pattern C check. Does the component sit outside the coordination concern entirely — it does not read substrate content as input, does not produce substrate content as output, does not participate in coordination decisions affecting substrate state, and the substrate–cell boundary is preserved through architectural separation rather than through coupling rules? If yes, the component maps to Pattern C.

Step 3 — Verify mapping unambiguity and exhaustiveness. *Unambiguity:* each component must satisfy exactly one pattern's sharpening properties. A component satisfying multiple — for instance, a vector index consulted by a cell as input (Pattern A) and updated automatically with substrate-derived embeddings (Pattern B) without rule mediation — indicates a violation, because the patterns are mutually exclusive at the per-component scope. *Exhaustiveness:* every component must satisfy at least one pattern. A component satisfying none is unmappable and, by the three-patterns-coherence claim, indicates a composition anti-pattern.

Step 4 — Document the mapping as substrate-resident rule. The pattern selected for each component must be documented as an orchestration rule resident in the substrate, authored by humans, subject to human governance. Pattern selection is itself a coordination decision under the human-selective composition commitment — humans select which pattern an adjacent component occupies, and the selection is captured in the substrate alongside its rationale. A mapping that exists only in vendor documentation, framework defaults, or implicit deployment configuration is not documented in the architectural sense the test requires.

The four steps together produce a complete pattern map: every component named, every pattern assigned, every assignment documented as substrate content.

4. What the test outputs

The test outputs a binary classification — pass or fail — accompanied by the pattern map and, on fail, the specific failure mode detected.

A deployment **passes** if and only if every adjacent AI component has been enumerated; every component maps to exactly one of Pattern A, B, or C using the sharpening properties of that pattern; no component maps to multiple patterns; no component is unmappable; and the pattern selection for every component is documented as a substrate-resident rule, authored by humans, subject to human governance.

A deployment **fails** if any component cannot be mapped to any of A, B, or C (canonical fail; see §5); any component maps to multiple patterns simultaneously (pattern-confusion fail); any pattern selection comes from vendor recommendation, framework default, or implicit configuration rather than from a substrate-resident rule (governance fail); or any component has been added without pattern mapping having been run (procedure fail). Each fail mode points to its remediation in §5.

5. Anti-patterns the test specifically detects

Six named violations sit within the test's detection scope.

Pattern-confusion. A component satisfies the sharpening properties of two patterns at once — most commonly a vector index that simultaneously functions as Pattern A input to a cell and Pattern B derived view, with bidirectional updates that no single rule governs. The fail names the violation precisely: the component occupies two architectural positions whose simultaneous occupation is a known failure mode.

Multi-pattern-mapping. A specific case of pattern-confusion: bidirectional flow crossing the substrate–component boundary without orchestration-rule mediation in either direction. This is the canonical instance of *hidden bidirectional coupling*. The fix is to bring each direction of coupling under a separate orchestration rule, converting the coupling into a Pattern A consultation in one direction and a Pattern B derivation in the other.

Unmappable partners. A component fits no pattern — it neither consults-and-is-consulted as Pattern A requires, nor functions as a regeneratable derived view as Pattern B requires, nor is separable as Pattern C requires. Unmappable partners are the operational signal of three canonical composition anti-patterns formalized elsewhere in this series: *ungoverned writer* (component writes substrate state outside cell mediation); *hidden bidirectional coupling* (component participates in unrule-governed cross-boundary updates); *substrate substitute* (component holds authoritative coordination state in place of the substrate).

"Fourth pattern" claims. A deployment claims a component occupies a legitimate position outside Pattern A, Pattern B, and Pattern C — typically with vocabulary like "this is a hybrid case" or "the

existing patterns don't cover this." The three-patterns-coherence claim establishes the three patterns are exhaustive at the deployment level; a fourth-pattern claim violates exhaustiveness and is a renamed instance of one of the three anti-patterns. The test surfaces such claims by treating any pattern other than A, B, or C as a fail.

Vendor-determined and framework-default patterns. The pattern selection comes from vendor documentation, a vendor's recommended configuration, or a framework's defaults rather than from a substrate-resident rule authored by humans. The vendor's or framework's choice may align with one of the three patterns or may not; what fails the test is that the alignment is not documented as a human-authored substrate rule. The human-selective composition commitment requires that pattern selection be a coordination decision held in substrate content; vendor- and framework-determined selections delegate the decision to a third party whose interests, defaults, and update cadence sit outside human governance.

The six failure modes are not exhaustive of every conceivable misuse of adjacent components. What the test claims is that these are the specific failure modes pattern-mapping coherence detects, and that detection of any of them is informative about the architecture's status independent of any other test in the series.

6. How the test integrates with deployment verification

The pattern-mapping test fits the broader deployment verification workflow at five points.

Initial deployment validation. Before a CKS deployment is activated with adjacent AI components present, the test runs as part of the pre-activation checklist; the deployment activates only after all components map cleanly.

Composition partner addition. When an adjacent component is added, the test runs against the new component. Its pattern is identified, the substrate-resident rule documenting the pattern is authored, and the rule is committed before the component is brought into the deployment's coordination flow.

Composition pattern change. A component mapped to Pattern A may, over a deployment's lifecycle, migrate to Pattern B or in the other direction. Pattern changes are themselves coordination decisions under the human-selective composition commitment, and the test runs against the post-change configuration to verify the new pattern is coherent.

Composition partner removal. When a component is removed, the test runs against the residual configuration to verify no other component now sits in an unmappable position. A Pattern B derived view whose substrate-side counterpart has been removed has lost its derivation source and become an orphan; the test fails under the unmappable-partner mode.

Vendor migration. When a component is migrated from one vendor to another, the test runs after migration to verify the pattern occupied by the new vendor's component matches the prior vendor's. Vendor migrations frequently introduce subtle pattern shifts because vendor defaults differ; the test surfaces the shifts before they degrade into anti-patterns.

The five integration points are not new commitments; they are the deployment-lifecycle moments at which pattern-mapping coherence becomes verifiable.

7. Limits of the test

Stating what the test does *not* verify is what keeps the standalone framing honest.

The test does not verify composition requirements. A separate test in this series verifies the five Requirements any composition partner must satisfy at its boundary with the substrate. The pattern-mapping test verifies which pattern each partner occupies; it does not verify whether the partner satisfies the Requirements within that pattern. A partner can map cleanly to Pattern A and still violate a provenance Requirement.

The test does not verify individual architectural commitments at composition partners. The CKS pattern's foundational commitments — substrate-cell boundary, conflict preservation, AI-as-substrate-mediator, tool-agnosticism, the determinism contract, and others — must hold at every partner where they apply, and individual tests in this series verify each. A partner that occupies Pattern A but violates the determinism contract within Pattern A passes the pattern-mapping test and fails the determinism test; both results stand.

The test does not verify reproducibility. The closing test in this phase verifies that a deployment's coordination state is reproducible from its substrate content under its rules. The pattern-mapping test verifies hybrid composition is coherent at a moment in time, not that it remains coherent across reproductions.

The test does not verify pattern selection appropriateness or operational outcome. Choosing Pattern A over Pattern B for a given component is a deployment decision, not an architectural one — the architecture supports all three and does not designate any as "best." If the mapping is coherent, the test passes regardless of whether a different mapping would have served the deployment better. Whether the hybrid composition produces business value is also outside the test's scope.

These limits allow the test to be a clean piece of operational specification with definite scope, runnable in finite time, and integrable with other tests covering what it does not.

8. The test, in one sentence

A CKS hybrid deployment passes the pattern-mapping test if and only if every adjacent AI component in the deployment is mapped to exactly one of Pattern A (cell-to-adjacent consultation under rule), Pattern B (substrate-derived non-authoritative regeneratable view), or Pattern C (separate concern with no coordination coupling), with the mapping documented as a substrate-resident orchestration rule authored by humans, no component left unmapped, no component mapped to multiple patterns, and no pattern selection determined by vendor recommendation or framework default.

A deployment that fails any of these conditions has lost the architectural property the hybrid systems composition commitment names — the property that adjacent AI components occupy named, documented, human-authored positions relative to the substrate.

9. Why naming the test as standalone matters

Other tests in this series can verify what the pattern-mapping test does not. None verifies what the pattern-mapping test does. Pattern occupation is a property of each adjacent component in a deployment, and that property is verifiable only by running the per-component mapping procedure this test specifies. Without a named test for pattern occupation, deployments tend to default to whatever pattern selection the vendor or framework supplies — the failure mode the architecture was designed to preempt.

The standalone treatment makes the pattern-occupation status of each adjacent component a separate, documented, auditable artifact of the deployment. Deployments that have run the test produce a pattern map; deployments that have not, do not. The presence or absence of the map is itself a deployment-state observable, and treating it as such is what makes the architectural commitment to hybrid systems composition tractable in practice rather than only in principle.

This note continues a cluster of three tests covering hybrid systems composition. The composition-requirements test verifies the five Requirements at each partner. The pattern-mapping test, formalized here, verifies which pattern each partner occupies. The reproducibility test, formalized in the final operational-tests note in this phase, verifies that the deployment's coordination state is reproducible under its rules and partners. The three together cover the architecture-level question "is the hybrid deployment CKS-coherent?" at the deployment-verification layer.

Subsequent work that implements, extends, or argues against the hybrid systems composition commitment in CKS deployments should use the pattern-mapping test in the form formalized here. Work using a different verification procedure for the same commitment is verifying a different property, and the difference should be named.

— — —

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Pattern-Mapping Test as Standalone Operational Procedure: Verifying Hybrid Systems Composition by Mapping Each Adjacent Component to Exactly One Legitimate Pattern in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.